MODULE 25

Introduction to Gen AI and LLM Models

Lectures Covered

Lecture 1 — AI vs ML vs DL vs GenAI
Lecture 2 — How Transformers Were Introduced for ChatGPT
Lecture 3 — OpenAI ChatGPT Model Training — Initial Steps
Lecture 4 — OpenAI ChatGPT Training — Final RLHF Step
Lecture 5 — Evolution of Large Language Models
Lecture 6 — All LLM Models Analysis

Lecture 1: AI vs ML vs DL vs GenAI

These four terms describe nested subfields — each more specific than the last. Understanding the hierarchy clarifies where GenAI and LLMs fit in the broader landscape.

The Hierarchy

Field	Definition | Examples
Artificial Intelligence (AI)	Any technique enabling machines to mimic human intelligence | Rule-based systems, expert systems, search algorithms, chess engines
Machine Learning (ML)	AI systems that learn from data without explicit programming | Linear regression, decision trees, SVM, random forests, XGBoost
Deep Learning (DL)	ML using neural networks with many layers; learns representations automatically | CNNs, RNNs, LSTMs, Transformers
Generative AI (GenAI)	DL models that generate new content — text, images, audio, code, video | GPT-4, DALL-E 3, Stable Diffusion, Sora, Whisper

What Makes GenAI Different from Discriminative AI?

Type	Task | Output | Examples
Discriminative AI	Classify or predict from existing data | Label, number | Spam filter, fraud detection, image classifier
Generative AI	Create new data that resembles training data | Text, image, audio, code | ChatGPT, Midjourney, Copilot

GenAI Modalities

- **Text:** GPT-4, LLaMA 3, Claude, Gemini — chatbots, writing, code, summarisation
- **Image:** DALL-E 3, Stable Diffusion, Midjourney — image generation and editing
- **Audio:** Whisper (speech->text), MusicGen, ElevenLabs (text->speech)
- **Code:** GitHub Copilot, CodeLLaMA, StarCoder — code completion and generation
- **Video:** Sora (OpenAI), Runway Gen-2 — text-to-video generation
- **Multimodal:** GPT-4o, Gemini 1.5 — process and generate across text, image, audio together

Lecture 2: How Transformers Were Introduced for ChatGPT

ChatGPT is built on the Transformer architecture, specifically a decoder-only variant. Understanding why the Transformer replaced RNNs — and how it became the backbone of all modern LLMs — is essential context.

Why RNNs Were Replaced

RNN Limitation	Impact | Transformer Solution
Sequential processing	Cannot parallelise — slow to train on long text | All tokens processed in parallel
Vanishing gradients	Forgets context beyond ~20-50 tokens | Direct O(1) attention to any token
Fixed-size context	Encoder compresses entire input into one vector | Attends to all encoder states
Slow on GPUs	GPUs excel at parallel operations; RNN is sequential | Attention is matrix multiplication — GPU-optimal

Transformer Key Components (Recap for ChatGPT Context)

Decoder-only Transformer (GPT architecture)

Input tokens -> Embedding + Positional Encoding
N x [Masked Self-Attention -> Add & Norm -> FFN -> Add & Norm]
Linear (LM head) -> Softmax -> probability over vocabulary
Autoregressive: generate one token at a time, feed back as next input
Causal mask ensures token t can only attend to tokens 1 through t.

Why decoder-only for ChatGPT?

BERT (encoder-only) is great at understanding but cannot generate text.
T5 (encoder-decoder) generates but is heavier — two large components.
GPT (decoder-only) uses one unified stack for both understanding and generation.
Scaling laws showed: bigger decoder = better generation. OpenAI scaled GPT.

Lecture 3: OpenAI ChatGPT Training — Initial Steps

ChatGPT was not just scaled up — it was carefully trained through three sequential stages. The first two stages produce InstructGPT, the direct precursor to ChatGPT.

Stage 1 — Pre-Training (Foundation)

Train a large GPT-3 decoder on ~570 GB of filtered web text, books, and code using next-token prediction. This creates a powerful completion engine but NOT an instruction-follower.

Pre-Training Detail	Value
Dataset	Common Crawl, WebText2, Books, Wikipedia, code (~570 GB filtered)
Objective	Causal LM: predict token t+1 given tokens 1..t
Parameters	GPT-3: 175B | GPT-3.5 (davinci-003): ~175B with code training
Compute	~3,640 petaflop/s-days on V100 GPUs (~$4.6M estimated cost)
Result	Strong text completer — continues any prompt, not instruction-aware

Stage 2 — Supervised Fine-Tuning (SFT)

Human labellers (OpenAI contractors) wrote ideal responses to a diverse set of prompts. The pre-trained model is fine-tuned on these (prompt, ideal_response) pairs.

```
# SFT training format
# Each example is a (prompt, response) pair

examples = [
    {'prompt': 'Explain photosynthesis simply.',
     'response': 'Photosynthesis is how plants make food from sunlight...'},
    {'prompt': 'Write a Python function to reverse a string.',
     'response': 'def reverse_string(s):\n    return s[::-1]'},
    {'prompt': 'What is the capital of France?',
     'response': 'The capital of France is Paris.'},
]

# Fine-tune on ~13K prompt-response pairs
# Same cross-entropy loss as pre-training but on curated instruction data
# Result: model that follows instructions — but still not well-aligned
```

Why SFT alone is not enough

SFT teaches format (follow instructions) but not values (be helpful, harmless, honest).
SFT model may still give confident wrong answers, harmful content, or sycophantic replies.
RLHF (Stage 3) corrects this by optimising for human preference, not just imitation.

Lecture 4: OpenAI ChatGPT Training — Final RLHF Step

Reinforcement Learning from Human Feedback (RLHF) is what transforms InstructGPT into ChatGPT. It uses human preference rankings to train a reward model, then optimises the LLM against that reward using PPO.

Stage 3 — RLHF in Three Sub-Steps

Sub-Step	Process | Goal
3a. Collect comparisons	SFT model generates 4-9 responses per prompt; human labellers rank them best to worst | Build preference dataset
3b. Train Reward Model (RM)	Train a separate model (typically GPT-3 size) to predict which response humans prefer | RM outputs a scalar score for any (prompt, response) pair
3c. PPO Fine-Tuning	Use Proximal Policy Optimisation (RL algorithm) to update the SFT model to maximise RM score, penalised by KL divergence from SFT | Policy learns to generate high-reward (human-preferred) responses

The RLHF Objective

```
# The optimisation objective at each step:

# reward = RM(prompt, response)   <- what humans prefer
# kl_pen = KL( policy || sft_model )  <- don't drift too far from SFT

# Total reward = reward - beta * kl_pen
#
# beta controls the KL penalty strength (typically 0.02 - 0.2)
# Without KL penalty: model would 'hack' the reward model (reward hacking)
# With KL penalty: model stays close to SFT but improves alignment

# PPO (Proximal Policy Optimisation):
# Clips the policy update ratio to [1-eps, 1+eps]  (eps ~ 0.2)
# Prevents catastrophically large updates in one step
```

Why RLHF Produces Aligned Behaviour

RLHF effects on ChatGPT

Helpfulness: trained to give complete, useful answers rather than terse completions.
Harmlessness: reward model penalises harmful/offensive outputs -> model refuses them.
Honesty: ranked responses show labellers prefer admitting uncertainty over false confidence.
Format: labellers prefer structured, readable responses -> model learns markdown, lists, etc.

RLHF Component	Role | Size
SFT Model (policy)	Starting point for RL; the model being trained | 175B params
Reward Model (RM)	Scores (prompt, response) pairs for human preference | ~6B params
Reference Model	Frozen SFT copy; used to compute KL penalty | 175B params
PPO Optimiser	Updates policy to maximise reward - beta*KL | Algorithm, no extra params

Reward hacking risk

If the RM is imperfect, the policy may find responses that fool the RM but aren't truly good.
Example: very long responses may score higher even if they are verbose and unhelpful.
Mitigated by: KL penalty, periodic human evaluation, and iterative RM retraining.

Lecture 5: Evolution of Large Language Models

The LLM landscape evolved rapidly from 2017 to today. Each generation solved specific limitations of the previous one — from the Transformer paper itself through to multi-modal reasoning models.

Timeline of Key Milestones

Year	Model	Breakthrough
2017	Transformer (Google)	Self-attention replaces RNNs; 'Attention Is All You Need'
2018	GPT-1 (OpenAI)	Decoder-only pre-training + fine-tuning; 117M params
2018	BERT (Google)	Bidirectional encoder; MLM + NSP; transfer learning for NLU
2019	GPT-2 (OpenAI)	1.5B params; zero-shot task transfer; staged release due to misuse risk
2019	T5 (Google)	Text-to-text framework; every NLP task framed as seq2seq
2020	GPT-3 (OpenAI)	175B params; emergent few-shot learning; in-context learning
2021	Codex (OpenAI)	GPT-3 fine-tuned on code; powers GitHub Copilot
2022	InstructGPT (OpenAI)	GPT-3 + SFT + RLHF; first aligned instruction-following model
2022	ChatGPT (OpenAI)	InstructGPT served as chat interface; 100M users in 60 days
2023	LLaMA (Meta)	Open-weight 7B-65B; democratised LLM research
2023	GPT-4 (OpenAI)	Multimodal; ~1.8T MoE; top human-level on bar exam, MMLU
2023	LLaMA 2 (Meta)	Commercial use allowed; 7B-70B; RLHF-trained chat models
2023	Gemini 1.0 (Google)	Natively multimodal from training; Ultra surpasses GPT-4 on MMLU
2024	Gemini 1.5 Pro (Google)	1M token context window; long-doc and video understanding
2024	LLaMA 3 (Meta)	8B/70B; 128K context; trained on 15T tokens; matches GPT-3.5
2024	GPT-4o (OpenAI)	Omni: native text+audio+vision; faster and cheaper than GPT-4
2024	o1 / o3 (OpenAI)	Chain-of-thought reasoning before response; PhD-level STEM

Key Trends Across Generations

- Scale: parameter counts grew from 117M (GPT-1) to 1.8T+ (GPT-4) — scale drives capability.
- Alignment: pre-training alone was not enough; SFT + RLHF became standard post-2022.
- Open-weight: Meta's LLaMA series democratised access; fine-tuning became widespread.
- Multimodality: models now natively process text, images, audio, and video together.
- Long context: from 4K tokens (GPT-3) to 1M tokens (Gemini 1.5) in 4 years.

Lecture 6: All LLM Models Analysis

A structured comparison of the major LLMs across open-weight, proprietary, and specialised categories — covering architecture, access, strengths, and best use cases.

Proprietary LLMs (API Access)

Model	Provider | Context | Strengths | Best For
GPT-4o	OpenAI | 128K | Multimodal, fast, cheap | General-purpose chat, vision, API apps
o1 / o3	OpenAI | 200K | Deep reasoning, STEM | Maths, science, complex logic
Claude 3.5	Anthropic | 200K | Long-doc, safe, honest | Analysis, writing, coding
Gemini 1.5 Pro	Google | 1M | Longest context, video | Long docs, video Q&A, multimodal
Gemini 1.5 Flash	Google | 1M | Fast, cheap, 1M ctx | High-volume production tasks
Command R+	Cohere | 128K | RAG-optimised | Enterprise search and retrieval

Open-Weight LLMs (Self-Hosted)

Model	Size | Context | Strengths | Best For
LLaMA 3 8B	8B | 128K | Small, fast, strong | Local deployment, edge devices
LLaMA 3 70B	70B | 128K | Near GPT-3.5 quality | Private cloud, high-quality generation
Mistral 7B	7B | 32K | Efficient, sliding window attention | Fast inference, limited hardware
Mixtral 8x7B	46.7B MoE | 32K | MoE: cheap inference | High quality at low cost
Gemma 2 9B/27B	9B/27B | 8K | Google-quality, compact | Research, fine-tuning
Phi-3 Mini	3.8B | 128K | Tiny but capable | Mobile, on-device, low-resource
CodeLLaMA 34B	34B | 100K | Code-optimised | Code generation, completion, debugging

Specialised / Domain LLMs

Model	Domain | Base | Key Use Case
Med-PaLM 2	Medical | PaLM 2 | Clinical Q&A, medical reasoning
BioGPT	Biomedical | GPT-2 | Biomedical literature mining
BloombergGPT	Finance | BLOOM | Financial news, sentiment, NER
StarCoder 2	Code | Custom | Multi-language code generation
Whisper (large)	Speech | Encoder-only | Multilingual speech-to-text
DALL-E 3	Image Gen | Diffusion + GPT-4 | Text-to-image, editing
Sora	Video Gen | Diffusion | Text-to-video, world simulation

How to Choose an LLM

Requirement	Recommended Choice
Best overall quality	GPT-4o or Claude 3.5 Sonnet
Best reasoning / math	o1 or o3 (OpenAI)
Longest context (doc/video)	Gemini 1.5 Pro (1M tokens)
Open-weight, high quality	LLaMA 3 70B or Mixtral 8x7B
On-device / edge deployment	Phi-3 Mini or Gemma 2 2B
Private data, self-hosted	LLaMA 3 8B or Mistral 7B via Ollama
Code generation	GPT-4o or CodeLLaMA 34B
RAG / enterprise search	Command R+ or Gemini 1.5 Flash
Domain-specific (medical/fin)	Fine-tune LLaMA 3 on domain data or use specialised model

Running Open-Weight LLMs Locally

```
# Ollama — easiest way to run LLMs locally
# Install: https://ollama.com

ollama pull llama3        # download LLaMA 3 8B
ollama run llama3         # interactive chat
ollama pull mistral       # Mistral 7B
ollama pull phi3          # Phi-3 Mini (3.8B)

# Use via Python (OpenAI-compatible API)
from openai import OpenAI
client = OpenAI(base_url='http://localhost:11434/v1', api_key='ollama')
response = client.chat.completions.create(
    model    = 'llama3',
    messages = [{'role': 'user', 'content': 'Explain transformers briefly.'}]
)
print(response.choices[0].message.content)

# HuggingFace local inference
from transformers import pipeline
pipe = pipeline('text-generation', model='microsoft/Phi-3-mini-4k-instruct',
                device_map='auto')
print(pipe('Explain attention mechanism:', max_new_tokens=100)[0]['generated_text'])
```

End of Module 25 Notes — Generative AI Course